- Python (http://mng.bz/Pz2Y) is also in the officially supported languages for auto-instrumentation.
- .NET (https://opentelemetry.io/docs/instrumentation/net/automatic/) supports auto-instrumentation as a .NET profiler to inject additional instrumentations at runtime, using monkey patching.
- Go (https://github.com/keyval-dev/opentelemetry-go-instrumentation) is a user contribution, based on eBPF. Strictly speaking, eBPF is a Linux syscall and Virtual Machine (VM), with certain requirements regarding the kernel version and symbol support, allowing you to push down user space functionality into the Linux kernel, triggering it based on events. In this context, it means to automatically capture telemetry signals for you.

Going forward, you can expect auto-instrumentation support for more programming languages as well as supporting metrics. Auto-instrumentation is a good baseline to always consider, since you get some insights essentially for free. However, it has its limitations: auto-instrumentation treats the service as a black box, meaning you won't get business-logic-related signals out of it. For this, you'd need to use manual instrumentation, so let's have look at it.

MANUAL INSTRUMENTATION

Manual instrumentation means that you modify your own code to manually emit signals. The steps required to do so, language agnostically, are

1. Import OpenTelemetry in your code—that is, take on a dependency on the language-specific API and the SDK.
2. For traces and metrics, first, create a provider (usually, there are defaults implemented), which, in turn, allows you to create an instance (along with a name and version).
3. For services (in contrast to a library you may instrument), in addition, configure the SDK, with appropriate options for exporting your signals.
4. After all the setup is complete, focus on the main job: create events (both traces and metric).

> **NOTE** Logs (http://mng.bz/JgrK) don't follow the previously shown pattern. Instead, they leverage existing setups; in other words, while OpenTelemetry takes a clean-slate approach for metrics and traces (via specifying new APIs and providing full implementations of said APIs across the programming languages), the approach with logs is different, in that the focus is on supporting existing logs and logging libraries.

There are many possible use cases and combinations of manual and auto-instrumentation, and if you're, indeed, looking for hands-on guidance on how to use this in the context of your setup, you will find a wide array of excellent articles and tutorials on the web. For example, "OpenTelemetry and Python: A Complete Instrumentation Guide"